5G / Mobile Analysis Track Write-up

5G-H1: The Quiet Exfil

1. What I tried first, and why it did not work

The challenge said the payloads were clean, so there was nothing to find inside the messages themselves. The hints pointed to network steganography, which meant the data had to be hidden in the packet headers rather than the payload. The setup was simple enough that I did not go through several failed attempts before settling on that.

2. The insight that actually cracked it

I knew the attacker was hiding 4 bits of data per packet. I wrote a Python script with Scapy that strips the GTP-U encapsulation and groups the packets into flows. For each flow I scanned the upper and lower nibbles (half-bytes) of the IP and UDP header fields, looking for the hex sync word "dead". It showed up in the lower nibbles of one specific flow, so the hidden message had to start right after it. I then took the nibbles that followed, paired them into 8-bit bytes, and decoded them as ASCII to get the flag.

3. How I satisfied myself the answer was right

The procedure was fairly direct, so the decoded output was my main check. Joining the extracted nibbles produced a readable ASCII string containing an RFCTF flag instead of random bytes, which showed I had found the covert channel and was not just reading noise.

4. Where I used an AI assistant and what I had to correct

I had not used Scapy before, so I used an AI assistant for the coding side. I described my approach (group by flow, scan the nibbles) and it gave me the Scapy calls to pull out the headers. I did not need to correct much, because the logic was already mine.

5G-H2: Replay

1. What I tried first, and why it did not work

My first idea was to scan the capture for duplicate sequence numbers. That does not work on its own, because the 5G network drops packets and legitimately retransmits them byte for byte, so a duplicate alone does not prove an attack.

2. The insight that actually cracked it

I needed a way to tell a quick, legitimate retry from a delayed, malicious replay. I wrote a Python script using the struct module to unpack the custom binary NASX headers carried over UDP. From each packet I pulled the UE ID, message type and sequence number, and built a chronological timeline for the subscriber in question. On that timeline, one packet stood out: it had been injected long after the phone's connection state had moved past that sequence number.

3. How I satisfied myself the answer was right

I compared the time gaps between each duplicate and its original. The legitimate retransmissions came shortly after a timeout. The packet I flagged arrived after a much longer gap, at a point where the connection had clearly advanced, so it was a replay and not an ordinary retry.

4. Where I used an AI assistant and what I had to correct

I used an AI assistant to help parse the custom binary headers. I had never used struct.unpack, so I told it which fields I needed (UE ID, sequence number and so on) and it helped me write the right format string. I only had to adapt the code to fit into my timeline script.

5G-H3: Needle in the Handovers

1. What I tried first, and why it did not work

The log was hard to read at first because I did not know the 5G terminology. The target's permanent ID (SUPI) appeared only once, so I filtered the log on the RAN ID attached to that event, assuming it identified the session. That failed. The network reuses RAN IDs, so my script merged the movements of unrelated users into the target's path and gave an impossibly large cell count.

2. The insight that actually cracked it

I asked an AI to explain the terms in the log. The key point was that 5G uses temporary identities (GUTIs) for privacy, so I could not follow the target with one fixed ID. I had to track the identities as they changed. I wrote a script that works as a small state machine: it watches for a realloc event to pick up the new GUTI, and when a release event ends the connection it saves that GUTI. If a later service event carries the same GUTI, that is the target reconnecting, so the script takes the new RAN ID from it and continues following the handovers.

3. How I satisfied myself the answer was right

With the plain RAN ID filter, the user appeared to jump between distant cells instantly. After I added the state tracking, those jumps disappeared and the path became a sequence of handovers between neighbouring cells with no impossible moves. That told me the tracking was following the right subscriber.

4. Where I used an AI assistant and what I had to correct

I knew very little about the 5G concepts in this challenge (SUPI, GUTI, RAN IDs), so I asked an AI assistant to explain what the question was asking and how temporary identities work. That led me to the state machine approach. The AI also helped structure the if/elif logic for the state variables, but it mixed up the service and register events in places, and I had to fix that logic myself.